

INF1103: Programming Fundamentals with DevOps

Week 10: Object Oriented Relationships Part 2

IMPORTANT

- This weeks slides ***SUCK!***
- Yes, they are very brief but don't worry!
- The lesson this week is very interactive so we won't give too much away on the uploaded slides
- The full set of uploaded slides with answers:
  - ***will be made available after the lesson***

Learning Outcomes

- Interaction and Interoperation
- Association
- Aggregation
- Composition
- Polymorphism

Interaction and Interoperation

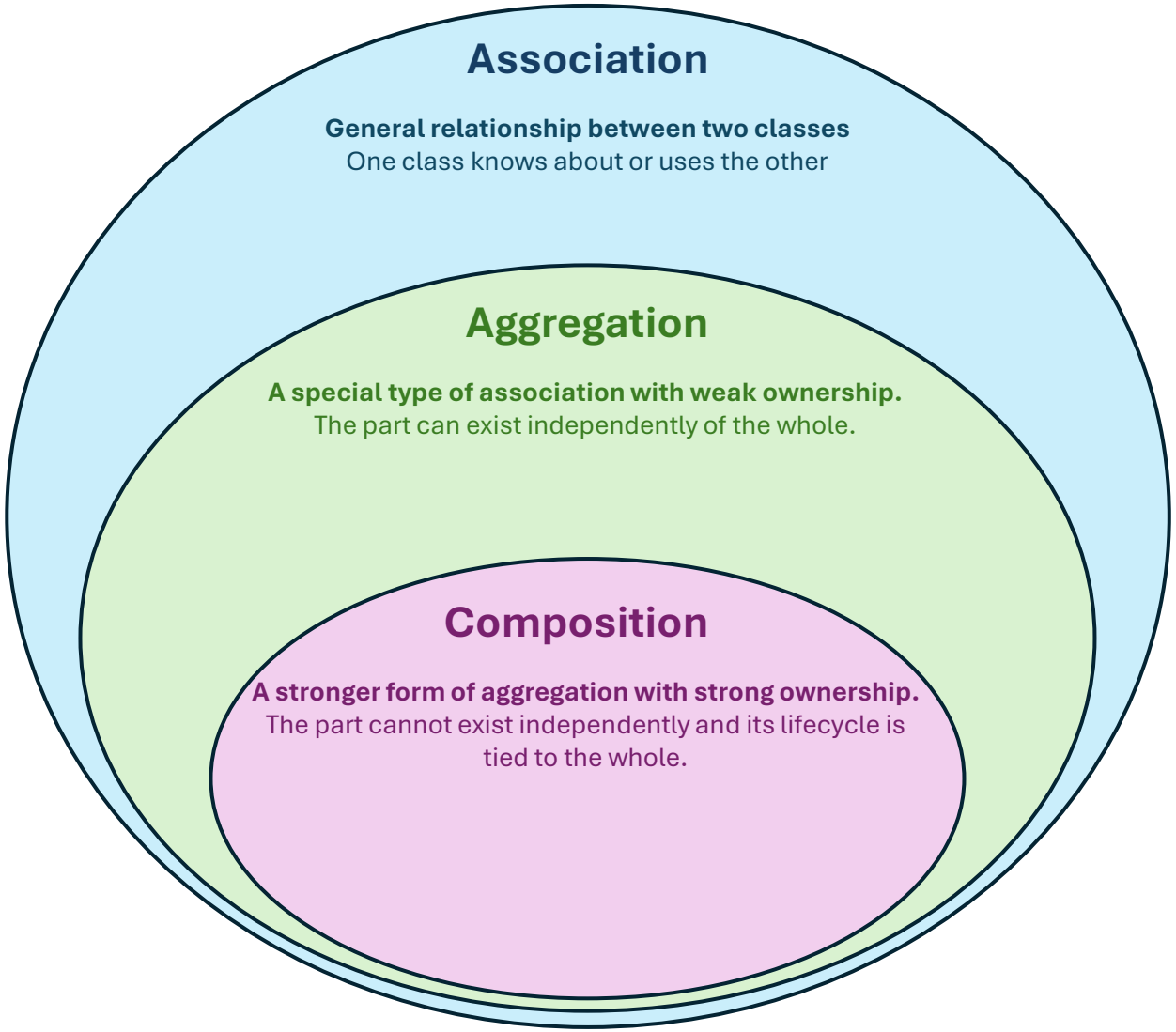
- How classes interact and interoperation determine how their relationships are implemented in code
- Classes can have tightly coupled or loosely coupled relationships
  - **Tightly coupled :** classes depend heavily on each other, so changes to one class may require changes to another
  - **Loosely coupled:** Classes have fewer dependencies with each other, making them easier to modify test and reuse independently.
- Generally, reusable and maintainable code benefits from loose coupling although some situations benefit from tight coupling
- Loosely coupled classes are ***easier*** to modify, test and reuse independently because they depend less on the internal implementation of other classes

Interaction and Interoperation

- This relationship between how classes are linked forms the basis for the relationships of:
  - Association
  - Aggregation
  - Composition

- And help us understand how each one is differentiated from the other.

Association / Aggregation / Composition



Polymorphism

- **Polymorphism** allows **different types of objects** or functions to be **handled through a single, common *interface***

- The word comes from the Greek ‘Poly’ meaning **many** and ‘morph’ meaning **forms**

- **Run-time Polymorphism (Method Overriding):** A child class provides its own specific implementation of a method already defined in its parent/base class. When you call the method through the base class reference, the program automatically runs the child class's version.

Polymorphism

- Other languages do this explicitly with type (Java example)
  - Animal animal1 = new Dog()
  - Animal animal2 = new Cat()

- The Animal class has a speak method() the Cat overrides with ‘Meow’ and the dog with ‘Woof’

- Animal1.speak() # ‘Woof’ is the output
- Animal2.speak() # ‘Meow’ is the output

Thanks!!!

